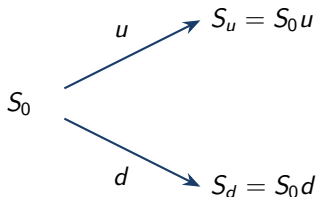


From a Financial Problem to C++ Code

FRE6883 – Recitation 1

One period, a tree, and a replicating hedge

One period: two possible stock prices



A European call at expiry

$$H_u = \max(S_u - K, 0)$$

$$H_d = \max(S_d - K, 0)$$

Example: $S_0 = 100$, $u = 1.2$, $d = 0.8$, $R = 1.1$, $K = 100$.

R is a gross growth factor: one risk-free dollar becomes R dollars.

Predict: in which state does the call pay something?

Explain the procedure to a computer

Pricing rule

$$q = \frac{R - d}{u - d}, \quad H_0 = \frac{qH_u + (1 - q)H_d}{R}.$$

q supplies the pricing weights; divide by R to express the result today.

Write the steps in plain English before writing any C++.

Assume $S_0 > 0$, $K \geq 0$, and $0 < d < R < u$.

Explain the procedure to a computer

Pricing rule

$$q = \frac{R - d}{u - d}, \quad H_0 = \frac{qH_u + (1 - q)H_d}{R}.$$

q supplies the pricing weights; divide by R to express the result today.

Define/read and check parameters $\rightarrow$ compute $S_u, S_d \rightarrow$ compute H_u, H_d

Compute $q \rightarrow$ compute $H_0 \rightarrow$ display the result.

The pricing weights sum to one; q is not a forecast of the up-state frequency.

Which variables and types do we need?

Choose a C++ type for prices, growth factors, and a step count.

Start with the inputs

$S_0 = 100$, $u = 1.2$, $d = 0.8$, $R = 1.1$, $K = 100$.

Would an integer type preserve every input?

Course convention: lower-case inputs in `main()`; S_0 , U , D , R in model functions.

Which variables and types do we need?

Choose a C++ type for prices, growth factors, and a step count.

Course notation in main()

```
double s0 = 100.0, k = 100.0;  
double u = 1.2, d = 0.8, r = 1.1;
```

Use `double` for the computed prices and probabilities too.

Use `int` for step counts and node indices.

Course convention: lower-case inputs in `main()`; S_0, U, D, R in model functions.

Compute the two stock prices

Complete the right-hand sides before the reveal.

```
double Su = /* ? */;
```

```
double Sd = /* ? */;
```

Code fragments in this part belong inside `int main()`.

Compute the two stock prices

Complete the right-hand sides before the reveal.

```
double Su = /* ? */;
```

```
double Sd = /* ? */;
```

Multiply the initial price by each factor

```
double Su = s0 * u;     $S_u = 120$ 
```

```
double Sd = s0 * d;     $S_d = 80$ 
```

Code fragments in this part belong inside `int main()`.

Implement the positive part

How can the program ensure that a call payoff is never negative?

```
double Hu = 0.0;
if (Su > k)
{
    Hu = Su - k;
}
```

Predict

What is H_u when $S_u = K$?

Would an else branch change the answer?

Alternative: `std::max(Su - k, 0.0)` needs `<algorithm>`.

We keep the professor's conditional-payoff style; repeat it for H_d .

Implement the positive part

How can the program ensure that a call payoff is never negative?

```
double Hu = 0.0;
if (Su > k)
{
    Hu = Su - k;
}
```

At the strike, the payoff is zero

Initialization already covers the other case. An explicit else could assign 0.0 again.

Alternative: `std::max(Su - k, 0.0)` needs `<algorithm>`.

We keep the professor's conditional-payoff style; repeat it for Hd.

Compute the price and display it

Which parentheses belong in the two missing expressions?

```
double q = /* (r - d) divided by (u - d) */;  
double optionPrice = /* weighted payoff divided by r */;
```

```
std::cout << std::fixed << std::setprecision(2)  
          << optionPrice << std::endl;
```

Compute the price and display it

Which parentheses belong in the two missing expressions?

```
double q = /* (r - d) divided by (u - d) */;  
double optionPrice = /* weighted payoff divided by r */;
```

Translate the formulas directly

```
double q = (r - d) / (u - d);  
double optionPrice = (q * Hu + (1.0 - q) * Hd) / r;
```

$q = 0.75$ and $H_0 = 15/1.1 \approx 13.6364$.

```
std::cout << std::fixed << std::setprecision(2)  
          << optionPrice << std::endl;
```

Exercise: complete the one-period pricer

Inside `main()`; headers and numeric inputs are supplied. Fill all blanks, then predict the printed result.

```
double s0 = 100.0, k = 100.0;
double u = 1.2, d = 0.8, r = 1.1;
double Su = /* A */, Sd = /* B */;
double Hu = 0.0, Hd = 0.0;
if (/* C */)
{
    Hu = /* D */;
}
if (/* E */)
{
    Hd = /* F */;
}
double q = /* G */;
double optionPrice = /* H */;
std::cout << std::fixed << std::setprecision(2)
    << optionPrice << std::endl;
```

Solution: a complete one-period program

Read the left column, then continue on the right. Inputs satisfy $S_0 > 0$, $K \geq 0$, $0 < d < R < u$.

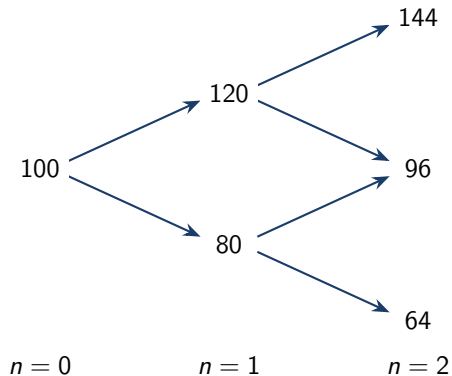
```
#include <iostream>
#include <iomanip>
int main()
{
    double s0 = 100.0, k = 100.0;
    double u = 1.2, d = 0.8, r = 1.1;
    double Su = s0 * u, Sd = s0 * d;
    double Hu = 0.0, Hd = 0.0;
    if (Su > k)
    {
        Hu = Su - k;
    }

    if (Sd > k)
    {
        Hd = Sd - k;
    }

    double q = (r - d) / (u - d);
    double optionPrice =
        (q * Hu + (1.0 - q) * Hd) / r;
    std::cout << std::fixed
                << std::setprecision(2)
                << optionPrice << std::endl;
    return 0;
}
```

Output: 13.64. **Check:** if $K = 120$, both payoffs and the price are zero.

How do we avoid writing every node manually?



Predict: how many distinct stock-price nodes are there at time 2?

One formula describes every stock-price node

$$S(n, i) = S_0 u^i d^{n-i}, \quad 0 \leq i \leq n.$$

n : number of elapsed steps. i : number of up moves.

There are $n - i$ down moves and $n + 1$ distinct nodes at time n .

`s0 * std::pow(u, i) * std::pow(d, n - i)`

`std::pow` comes from `<cmath>`; n and i are integers.

Quick check

What are (n, i) and the stock price after one up move and one down move?

One formula describes every stock-price node

$$S(n, i) = S_0 u^i d^{n-i}, \quad 0 \leq i \leq n.$$

n : number of elapsed steps. i : number of up moves.

There are $n - i$ down moves and $n + 1$ distinct nodes at time n .

```
s0 * std::pow(u, i) * std::pow(d, n - i)
```

`std::pow` comes from `<cmath>`; n and i are integers.

Two steps, one up move

$(n, i) = (2, 1)$, so $S(2, 1) = 100(1.2)(0.8) = 96$.

Exercise: complete the nested loops

```
const int N = 2;
for (int n = 0; n <= N; n++)
{
    for (int i = /* ? */; /* ? */; /* ? */)
    {
        std::cout << n << " " << i << " "
                    << /* stock price */ << std::endl;
    }
}
```

Explain your bounds:

- What do n and i represent? Why must $i \leq n$?
- How many nodes are printed at time n ? How many in total for $N = 2$?

Solution: visit each node once

```
const int N = 2;
for (int n = 0; n <= N; n++)
{
    for (int i = 0; i <= n; i++)
    {
        std::cout << n << " " << i << " "
                    << s0 * std::pow(u, i)
                    * std::pow(d, n - i) << std::endl;
    }
}
```

Check the traversal

n selects the time level; $i = 0, \dots, n$ visits its $n + 1$ nodes.

For $N = 2$: $1 + 2 + 3 = 6$ nodes. At $n = 2$, prices print as 64, 96, 144.

When will we need to keep a computed value?

Printing a stock price is enough to inspect it. Pricing backward requires the next level's option values again.

```
const int SIZE = 9;  
double Price[SIZE] = { 0.0 };
```

An array is an indexed row of values

`Price[0]`, `Price[1]`, ..., `Price[8]` are nine double slots.

The valid indices are 0 through 8; the array does not check the bounds for us.

For $N = 2$: terminal call payoffs are $[0, 0, 44]$.

Ask: do we need to store the entire tree to reuse one level?

We will keep one level in `Price`; Deck 2 will revisit array syntax.

A hedge must match both option payoffs

Replicate one long call

Hold Δ shares and invest B dollars in the risk-free asset today.
At the next step, the cash position is worth BR .

$$\Delta S_u + BR = H_u,$$

$$\Delta S_d + BR = H_d.$$

Ask: what disappears if we subtract the second equation from the first?

$B < 0$ means borrowing. The initial cost is $\Delta S_0 + B$.

Delta is the number of shares in the hedge

$$\Delta(S_u - S_d) = H_u - H_d \implies \boxed{\Delta = \frac{H_u - H_d}{S_u - S_d}}.$$

Interpret the ratio

The option's change between states, divided by the stock's change, gives the number of shares needed to replicate one option.

Then compute the cash position and the cost:

$$B = \frac{H_d - \Delta S_d}{R}, \quad H_0 = \Delta S_0 + B.$$

Predict: if both call payoffs are zero, what is Δ ?

To hedge a short call, hold this replicating portfolio.

Exercise: compute and interpret the hedge

Use the same inputs: $S_0 = 100$, $u = 1.2$, $d = 0.8$, $R = 1.1$, $K = 100$.

1. Compute S_u and S_d .
2. Compute H_u and H_d .
3. Compute q and H_0 .
4. Compute Δ .
5. Explain what Δ means.

Useful formulas

$$q = (R - d)/(u - d)$$

$$H_0 = (qH_u + (1 - q)H_d)/R$$

$$\Delta = (H_u - H_d)/(S_u - S_d)$$

If finished: find B and check the portfolio in both states.

Solution: half a share and a cash position

Price and delta

$$S_u = 120, S_d = 80$$

$$H_u = 20, H_d = 0, q = 0.75$$

$$H_0 = 15/1.1 \approx 13.6364$$

$$\Delta = (20 - 0)/(120 - 80) = 0.5$$

Check replication

$$B = (0 - 0.5 \cdot 80)/1.1$$

$$B \approx -36.3636, \text{ so } BR = -40.$$

$$\text{Up: } 0.5 \cdot 120 - 40 = 20.$$

$$\text{Down: } 0.5 \cdot 80 - 40 = 0.$$

$$\text{Cost: } 0.5 \cdot 100 + B \approx 13.6364.$$

Interpretation: hold half a share per call and borrow about 36.36 today.

Compute with full precision; round only the displayed result.

Design a function that computes delta

Before seeing the function, decide:

1. What is the return type?
2. Which four arguments are needed?
3. Does the function need to modify any of them?
4. Should these parameters be passed by value or reference?
5. Which expression should be returned?

Just enough function vocabulary

Arguments supply inputs; `return` sends back the result.

By value gives the function a copy. A reference names the caller's variable.

Use the name `CalculateDelta`, consistent with `CalculateAssetPrice`.

Solution: return one double from four inputs

```
namespace fre {  
    double CalculateDelta(double Su, double Sd,  
                           double Hu, double Hd)  
    {  
        return (Hu - Hd) / (Su - Sd);  
    }  
}
```

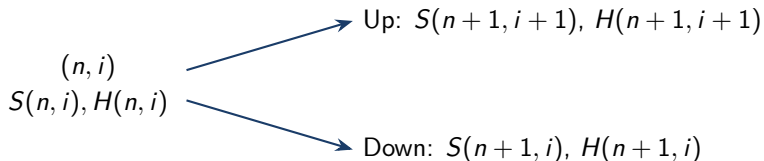
```
double delta = fre::CalculateDelta(Su, Sd, Hu, Hd);
```

By value is sufficient: these four numeric inputs do not need to change.

`fre::` selects the function in the course namespace. Define it before `main()` for a one-file program.

Assume $S_u > S_d$ before dividing.

The same hedge works locally in a CRR tree



$$\Delta(n, i) = \frac{H(n + 1, i + 1) - H(n + 1, i)}{S(n + 1, i + 1) - S(n + 1, i)}$$

Once the two next-period option values are known, we can compute the current hedge. Recompute it at each visited non-terminal node.

Ask: are these option values always terminal payoffs?

Only on the last step; earlier they are continuation values found by pricing backward.

Read one backward-induction update

At time n , the active array entries still hold time- $n + 1$ option values:

$$\text{Price}[i] = H(n + 1, i), \quad \text{Price}[i+1] = H(n + 1, i + 1).$$

```
Price[i] = (q * Price[i + 1]  
           + (1.0 - q) * Price[i]) / R;
```

After this line, $\text{Price}[i]$ holds $H(n, i)$ instead.

Predict: for our two-step example, terminal values $[0, 0, 44]$ become what at time 1?

Read one backward-induction update

At time n , the active array entries still hold time- $n + 1$ option values:

$$\text{Price}[i] = H(n + 1, i), \quad \text{Price}[i+1] = H(n + 1, i + 1).$$

```
Price[i] = (q * Price[i + 1]  
           + (1.0 - q) * Price[i]) / R;
```

After this line, $\text{Price}[i]$ holds $H(n, i)$ instead.

Predict: for our two-step example, terminal values $[0, 0, 44]$ become what at time 1?

The active row becomes $[0, 30]$

$$H(1, 0) = 0, \text{ and } H(1, 1) = (0.75 \cdot 44) / 1.1 = 30.$$

Exercise: compute the hedge before losing data

```
for (int n = N - 1; n >= 0; n--)  
{  
    for (int i = 0; i <= n; i++)  
    {  
        // A: compute the two next-period stock prices  
        // B: compute and display delta  
        // C: replace Price[i] by the current option price  
    }  
}
```

Complete the algorithm in words or C++:

- Which option entries and stock nodes belong in the delta formula?
- Why must B come before C? Why visit *i* from left to right?

Trace at the root: old row [0, 30], stock children [80, 120].

What goes wrong if Price[0] is replaced first?

Solution: use both children, then overwrite

1. Compute $S_u = S(n+1, i+1)$ and $S_d = S(n+1, i)$.
2. Compute $\Delta = (\text{Price}[i+1] - \text{Price}[i]) / (S_u - S_d)$.
3. Display delta, then apply the usual pricing update to $\text{Price}[i]$.

Correct root calculation

Old option row: $[0, 30]$.

$$\Delta(0, 0) = (30 - 0) / (120 - 80) = 0.75.$$

$$\text{Then } H(0, 0) = 0.75 \cdot 30 / 1.1 \approx 20.4545.$$

If you overwrite first

$$(30 - 20.4545) / (120 - 80) \approx 0.2386.$$

This mixes values from two different time levels.

Increasing i preserves both needed children until use; decreasing i would overwrite the next child's value too early.

Prepare the terminal row in the existing pricer

Within `fre::PriceByCRR(S0, U, D, R, N, K)`: check inputs, then initialize.

```
const int SIZE = 9;
if (N < 1 || N >= SIZE
    || S0 <= 0.0 || K < 0.0
    || D <= 0.0 || U <= D
    || R <= D || R >= U)
{
    std::cerr << "Invalid model data"
               << std::endl;
    return -1.0;
}

double q = RiskNeutProb(U, D, R);
double Price[SIZE] = { 0.0 };
for (int i = 0; i <= N; i++)
{
    Price[i] = CallPayoff(
        CalculateAssetPrice(
            S0, U, D, N, i), K);
}
```

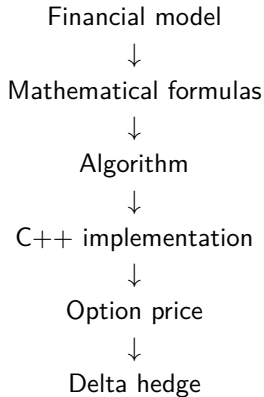
`SIZE = 9` supports $1 \leq N \leq 8$. The array holds one level.

`CallPayoff` computes the positive part; `CalculateAssetPrice` implements the stock-node formula.

Augment the existing CRR loop

```
for (int n = N - 1; n >= 0; n--)
{
    for (int i = 0; i <= n; i++)
    {
        double Su = CalculateAssetPrice(S0, U, D, n + 1, i + 1);
        double Sd = CalculateAssetPrice(S0, U, D, n + 1, i);
        double delta = CalculateDelta(Su, Sd, Price[i + 1], Price[i]);
        std::cout << "Delta(" << n << ", " << i << ") = "
                    << delta << std::endl;
        Price[i] = (q * Price[i + 1]
                    + (1.0 - q) * Price[i]) / R;
    }
}
return Price[0];
```

$N = 2$: $\Delta(1, 0) = 0$, $\Delta(1, 1) \approx 0.9167$, $\Delta(0, 0) = 0.75$; price ≈ 20.4545 .



One tree, two computations

The binomial tree gives both option values and the shares needed for the local replicating hedge.

Exit question: why must delta be computed before overwriting `Price[i]`?

Next: practise the C++ syntax that makes these steps work.